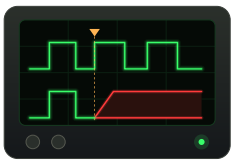


LibreILA (Integrated Logic Analyzer) Datasheet

Version: 0.0
Date: July 2026
Author: Yash Paritkar (github/yashparitkar)

1 Features

- Open-source and free to use: github.com/yashparitkar/libreila
- Generic and modifiable probe
- Uses fabric RAM for data storage and can be read back with AXI4Lite interface
- Uses series probes for ease with block diagram
- Customizable trigger:
  - Trigger can be configured with AXI4Lite
  - ARM can be configured with AXI4Lite
  - Adjustable trigger point
  - Level and edge based trigger
  - External trigger can be possible
  - Trigger can be chained across multiple instances
- CDC on the ARM and status bits
- A serial wrapper is also provided to easily use the ILA with PC
- A python driver is provided to configure the ILA over the serial wrapper and dump the capture as a VCD for any waveform viewer. A GUI on top of it is planned
- A default example is provided to use the ILA with a simple 64-bit AXI4Stream interface

2 Application

- RTL debugging
- FPGA prototyping
- Generic logic analyser

3 Description

LibreILA is an in-system Integrated Logic Analyzer (ILA) written in plain VHDL. It is spliced into a link inside the FPGA fabric and samples that link into a circular buffer held in fabric RAM, so nets that never reach a pad are as visible as the ones that do. It exists because the Microchip toolchain has no equivalent of the Xilinx ILA and SmartDebug's live probes are not a substitute for a triggered, time-correlated capture.

The core is deliberately ignorant of what it observes: one flat probe word of `G_PROBE_WIDTH` bits, whose bit order the trigger, the sample buffer and both drivers all inherit. A build is therefore described entirely by two CSV files. Two deliverables are provided: `libre_ila`, the bare core with an AXI4Lite slave port and a baremetal C driver, and `libre_ila_uart`, a serial wrapper driven from a PC by the python driver. The baremetal path is the differentiator: capture and readout need no host on the wire at all.

Contents

4 Revision History

Version	Date	Comments
0.0	25 July 2026	Document created.
0.1	4 August 2026	DISARM mapped onto the reserved input register at `0x2C`.

5 Detailed description

The summary on page 1 says what the core is. This section says how it works: what the blocks are, how a capture proceeds from arming to readout, and where the two clock domains meet. The register-level detail that a driver needs is in Section 7.

5.1 Overview

Everything in the core is arranged around one shared definition of the probe word, `w_probe`. Three blocks sit on top of it:

- **The probe splice.** A pair of mirrored ports shorted combinationally, so the probed link passes straight through and the core reads it in parallel.
- **The trigger comparator.** One condition bit and one mask bit per probed bit, reduced to a single condition and then qualified by the level or edge mode.
- **The sample buffer and the AXI4Lite slave.** A circular buffer in fabric RAM, written every sampling clock while armed and read back as a flat register map.

A capture runs in one direction. The host writes the trigger condition, the mask, the mode and the trigger position, then writes ARM_FT. The core starts sampling into the buffer immediately, overwriting the oldest sample once it wraps, so that at any moment the buffer holds the most recent `DEPTH` samples. When the trigger condition is met the core latches where it happened, keeps sampling for the configured number of post-trigger samples, and stops. DONE then goes high and the buffer holds a window of history either side of the trigger, which the host reads back at its leisure – the capture is over, so readout races nothing. A capture that has not got that far can be taken back at any point by writing DISARM, which returns the core to IDLE from either side of the trigger without disturbing a capture already finished.

The core straddles two clock domains and is deliberate about which side each part lives on. Sampling, triggering and the buffer write port run on `samp_aclk`, the clock of the probed link. The register map and the buffer read port run on `s_axil_aclk`. Only the arm and disarm pulses and the three status bits actually cross, through two-flop synchronisers; the sample data never crosses a synchroniser at all, because the buffer is a dual-port RAM whose read port is simply clocked from the other domain.

`libre_ila_uart` wraps all of the above with a UART and a packet parser, putting the same register map at the end of a serial link instead of on an AXI bus. Nothing inside the core changes.

TODO: Add the block diagram here.

Figure 1: LibreILA core block diagram

5.2 The probe

The core samples one vector, `w_probe`, of `G_PROBE_WIDTH` bits. Signals are packed LSB first in the order they are listed in `portmap.csv`, and that concatenation is the single definition of the probe bit order: the sample buffer, the trigger vector and both drivers inherit it. For the stock 64-bit AXI4Stream build the probe is 67 bits wide, laid out as in Table 1.

Bit range	Signal
63 downto 0	TDATA
64	TLAST
65	TVALID
66	TREADY

Table 1: Probe word of the stock AXI4Stream build

The core is spliced into the link it probes, so the probe appears as a pair of mirrored ports named after the role the core plays on each side, not after the direction the data happens to flow:

- `probe_slave_*` faces the *master* of the probed link, so the core is the slave there, and these ports carry the directions listed in `portmap.csv`.
- `probe_master_*` faces the *slave* of the probed link, so the core is the master there, and every direction is the mirror of its `probe_slave_` twin.

The two sides are shorted combinationally, so the splice is purely pass through and introduces no delay into the probed link. A signal that cannot be spliced in series is tapped in parallel instead, with the `mon` type, which the core reads and never drives.

5.3 Capture

While the ILA is armed it stores one sample of the probe word per rising edge of `samp_aclk` into a circular buffer `G_SAMP_BUFF_DEPTH` samples deep. Once the trigger fires the core keeps capturing for `DEPTH - TRIG_POS - 1` further samples and then stops, so the captured window holds `TRIG_POS` samples of history ahead of the trigger and the rest behind it.

Because the buffer is circular, the oldest sample is not at index zero and the window is almost always split across the wrap point. The core reports where the window starts and where the trigger landed in SAMP_BUFF_FRST_IDX and SAMP_BUFF_TRIG_IDX, and the host unrolls the buffer from there. Figure 2 shows the two views: the capture as it happened, and the same samples as they sit in the buffer.

TODO: Add the sample buffer position diagram here. The two views to draw are in the comment block above `trig_idx` in `hdl/libre_ila.vhdl`: the sample train split into `TRIG_POS` pre-trigger samples and the post-trigger remainder, and the same window inside the circular buffer, where wrapping puts a post-trigger run of a samples before `FRST_IDX` and another of b after `TRIG_IDX`.

Figure 2: Sample buffer positions

That split is also what the post-trigger counter inside the core counts out: it targets $a + b$ samples, not the distance to the end of the buffer.

5.4 Trigger

The trigger is one condition bit per probed bit, held in two registers of identical layout. TRIG_COND carries the value each probe bit has to take to match, TRIG_MASK decides whether that bit takes part at all. The enabled bits are then reduced to a single condition, by AND or by OR, and that single condition is what the level and edge modes act on (Table 2).

EDGE	FALLING	Mode	Triggers when the condition
0	x	Level	*is* true
1	0	Rising	*becomes* true
1	1	Falling	*becomes* false

Table 2: Trigger modes, `TRIG_CFG` bits 2 and 1

The distinction matters at arming time. In level mode a condition that already holds when the ILA is armed fires on the very first sample and captures nothing of interest, which is exactly what happens to "trigger when TVALID is high" on a mostly idle stream. The edge modes wait for a transition instead. The previous-condition flop is seeded with the live condition at arm rather than cleared, so an already-true condition is not mistaken for a rising edge.

Edge detection is applied to the reduced condition, not per probe bit, because by that point the whole probe word has already been reduced to one bit and the mode costs a single flip-flop. Asking for "bit 5 rises *and* bit 9 falls in the same sample" is therefore not expressible.

Note that an all-zero TRIG_MASK in AND mode matches vacuously and fires on the first sample after arming. Use OR mode with an all-zero mask if the intent is never to trigger on the probe.

5.5 External trigger and chaining

Setting `G_EXTERNAL_TRIG` to 1 replaces the trigger comparator with the `i_ext_trig` pin: in that build the ILA triggers only on the rising edge of that pin, synchronised into the sampling clock domain, and the three TRIG_CFG mode bits are ignored. The resulting trigger is driven out on `o_trig_out` in every build, so several instances can be chained to capture one event across different parts of a design.

5.6 Clock domains

The capture side runs entirely in the sampling clock domain, driven by `samp_aclk`, which should be fed the clock the probed link runs on. The AXI4Lite port is a separate domain on `s_axil_aclk`. The arm and disarm pulses and the ARMED, TRIGD and DONE status bits cross between them through two-flop synchronisers and so lag by a couple of cycles; the internal STATE field of the status register is not synchronised and is there for debugging only.

Arm and disarm cross on separate paths, each a write-toggled bit resynchronised and edge-detected on the sampling side. Two writes a few AXI cycles apart can therefore land in either order in the sampling domain, so a host that writes both back to back has to leave a gap between them if it cares which one wins.

Readout needs no synchroniser at all. The sample buffer is a dual-port LSRAM whose read port is clocked by the AXI4Lite clock, which is what the PolarFire LSRAM independent clock port exists for.

5.7 Readout

Samples are stored one 32-bit lane per slice of the probe word, so lane n carries probe bits $32n + 31$ downto $32n$ and the last lane is zero padded in its upper bits. On readout the lanes are padded further, up to `C_AXIL_STRIDE` registers per sample, so that a sample index maps onto an address with a shift instead of a multiply. The trigger vector uses that same padded layout, which is what makes a captured sample and a trigger condition directly comparable word for word.

6 Core parameters and interface signals

Everything in this section applies to both `libre_ila` and `libre_ila_uart`; the wrapper adds the generics and ports marked as such.

Three quantities are derived from the generics and are referred to throughout the rest of this document:

- `C_N_LANES` = $\lceil$`G_PROBE_WIDTH`/32$\rceil$, the number of 32-bit lanes one sample occupies in the RAM.
- `C_AXIL_STRIDE` (a), `C_N_LANES` rounded up to the next power of two with a minimum of 4. This is the number of registers one sample, and one trigger vector, occupies on the bus.
- `C_ADDR_WIDTH` = $\log_2$(`G_SAMP_BUFF_DEPTH`), the width of a sample index.

For the stock 64-bit AXI4Stream build `G_PROBE_WIDTH` is 67, so `C_N_LANES` is 3, a is 4 and `C_ADDR_WIDTH` is 11.

6.1 Configuration parameters

Generic	Default	Description
`G_SAMP_CLK_FREQ`	100 000 000	Sampling clock frequency in Hz. Reported over AXI4Lite so the host can build the time axis; it configures no logic.
`G_AXIL_CLK_FREQ`	100 000 000	AXI4Lite clock frequency in Hz. Sets the wrapper's watchdog period and the UART divider.
`G_EXTERNAL_TRIG`	0	1 uses the `i_ext_trig` pin instead of the trigger vector.
`G_PROBE_WIDTH`	67	Total probed bits. Keep it a multiple of 32 for best results. Must be greater than zero, checked at elaboration.
`G_SAMP_BUFF_DEPTH`	2048	Number of samples stored. Must be a power of two greater than one, checked at elaboration.
`G_UID`	0	Identity of this instance, reported at UID so that a system holding several cores can tell them apart. Configures no logic and every value is legal, zero meaning unset. A `natural`, so at most `0x7FFFFFFF`.
`G_UART_RX_FIFO_DEPTH`	1024	Wrapper only. Depth of the UART receive FIFO in bytes.
`G_UART_TX_FIFO_DEPTH`	1024	Wrapper only. Depth of the UART transmit FIFO in bytes.
`G_BAUD_RATE`	115 200	Wrapper only. UART baud rate.
`C_S_AXIL_DATA_WIDTH`	32	AXI4Lite data width. Do not change.
`C_S_AXIL_ADDR_WIDTH`	32	AXI4Lite address width. Do not change.

Table 3: Configuration generics

6.2 Portmap

The probe ports are not fixed. They are written by the generator from `portmap.csv`, one line per probed signal as `signal,width,type`, listed LSB first. The same file gives the python driver the names it puts in the VCD, so one file describes the probe for the hardware and for the host at once.

`type` is the direction the signal has on the `probe_slave_` side, that is the direction the probed link's slave would give it (Table 4). There is no `inout`: a signal assignment cannot pass a bidirectional line through, and an ILA belongs on the fabric side of the IO buffer where the bus is not bidirectional yet. Where a link genuinely cannot be spliced, tap it with `mon` instead.

Type	Slave side	Master side	Pass through
`in`	input	output	yes, slave to master
`out`	output	input	yes, master to slave
`mon`	input	none	no, parallel tap only

Table 4: Probe signal types in `portmap.csv`

The remaining ports are fixed and are listed in Table 5. The AXI4Lite slave port carries the standard AW, W, B, AR and R channel signals with the `s_axil_` prefix and is not enumerated here.

Port	Dir	Width	Description
`i_rst_sync`	in	1	Synchronous reset, active high
`samp_aclk`	in	1	Sampling clock, the clock of the probed domain
`i_ext_trig`	in	1	External trigger input, rising edge sensitive, used when `G_EXTERNAL_TRIG` is 1
`o_trig_out`	out	1	Trigger output, for chaining to further ILA instances
`s_axil_aclk`	in	1	AXI4Lite clock
`s_axil_aresetn`	in	1	Core only. AXI4Lite reset, active low; the wrapper drives it internally
`s_axil_*`	—	—	Core only. AXI4Lite slave channels, `C_S_AXIL_DATA_WIDTH` wide
`uart_rx`	in	1	Wrapper only. Serial receive
`uart_tx`	out	1	Wrapper only. Serial transmit
`probe_slave_*`	—	—	Probe port facing the master of the probed link
`probe_master_*`	—	—	Probe port facing the slave of the probed link, every direction mirrored

Table 5: Fixed interface signals

6.3 Core generation

A build is described entirely by two CSV files in `codegen/`: `portmap.csv` for the probe, and `configuration.csv` for the generic values, one line per generic as `generic,type,value`. Running the generator emits a complete set of VHDL sources for that configuration:

```
python3 codegen/code_generator.py \
        --portmap codegen/portmap.csv \
        --config  codegen/configuration.csv
```

Left at their shipped defaults, the two files describe the stock 64-bit AXI4Stream build. `GEN_TYPE` in the configuration selects what comes out: 0 emits the bare AXI4Lite core alone, 1 emits it together with the UART wrapper and the FIFO and UART blocks it instantiates. Add the emitted files to the project, instantiate the core or the wrapper, and splice the probe ports into the link to be observed. The full procedure is in the user manual; this section is a quick glance.

TODO: Add screenshots of the generator run and of the core instantiated in SmartDesign once the core is released.

7 Register summary

The core presents a flat map of 32-bit registers on its AXI4Lite slave port; a register's byte address is its index times four. The map is laid out in three blocks: the **output block** first, then the **input block**, then the **sample buffer**.

That order is deliberate. The output block is eight registers whatever the core was synthesised with, so it can sit at a fixed place. The two blocks above it move with the probe width, and the output block is what reports the probe width in the first place: putting it last would mean a host had to know the probe width in order to find the register that tells it the probe width. Both supplied drivers rely on this, reading the geometry out of the core at start-up and deriving the rest of the map from it, so re-synthesising with a different probe width needs no driver rebuild.

Three kinds of register appear in the map:

- **Output, read only.** Status, and the synthesis-time geometry of the core.
- **Input, read/write.** The trigger configuration, plus the write-only arm register.
- **Sample buffer, read only.** The capture itself, a registers per sample.

Table 6 lists the whole map. Offsets in the input block and above are given symbolically in a, with the value for the stock build ($a = 4$) in parentheses.

Offset	Name	Access	Reset	Description
Output block				
0x00	STATUS	R	0x00000000	ILA status and state
0x04	MGCKEY	R	0xB01DFACE	Magic key, identifies the core type
0x08	SAMP_CLK_FREQ	R	G_SAMP_CLK_FREQ	Sampling clock frequency in Hz
0x0C	WIDTH	R	G_PROBE_WIDTH	Total probed bits
0x10	DEPTH	R	G_SAMP_BUFF_DEPTH	Sample buffer depth in samples
0x14	UID	R	G_UID	Identity of this instance
0x18	SAMP_BUFF_TRIG_IDX	R	0x00000000	Buffer index of the trigger sample
0x1C	SAMP_BUFF_FRST_IDX	R	0x00000000	Buffer index of the oldest sample
Input block				
0x20	TRIG_POS	R/W	0x00000000	Number of pre-trigger samples
0x24	ARM_FT	W	0x00000000	Any write arms the ILA, or forces the trigger if it is already armed
0x28	TRIG_CFG	R/W	0x00000000	Trigger mode: AND/OR, level/edge, rising/falling
0x2C	DISARM	W	0x00000000	Any write cancels a capture in progress and returns the ILA to IDLE. Ignored once the capture is DONE
0x30 + 4n	TRIG_COND(n)	R/W	0x00000000	Trigger condition, word n, $n = 0 \ldots a-1$ (0x30–0x3C)
0x30 + 4n + 4a	TRIG_MASK(n)	R/W	0x00000000	Trigger mask, word n, $n = 0 \ldots a-1$ (0x40–0x4C)
Sample buffer				
0x30 + 8a + 4($ak + l$)	SAMP_BUFF(k,l)	R	—	Sample k, lane l, for $k = 0 \ldots$ DEPTH-1 and $l = 0 \ldots a-1$ (base 0x50)

Table 6: Register map. Offsets are from the AXI4Lite base address.

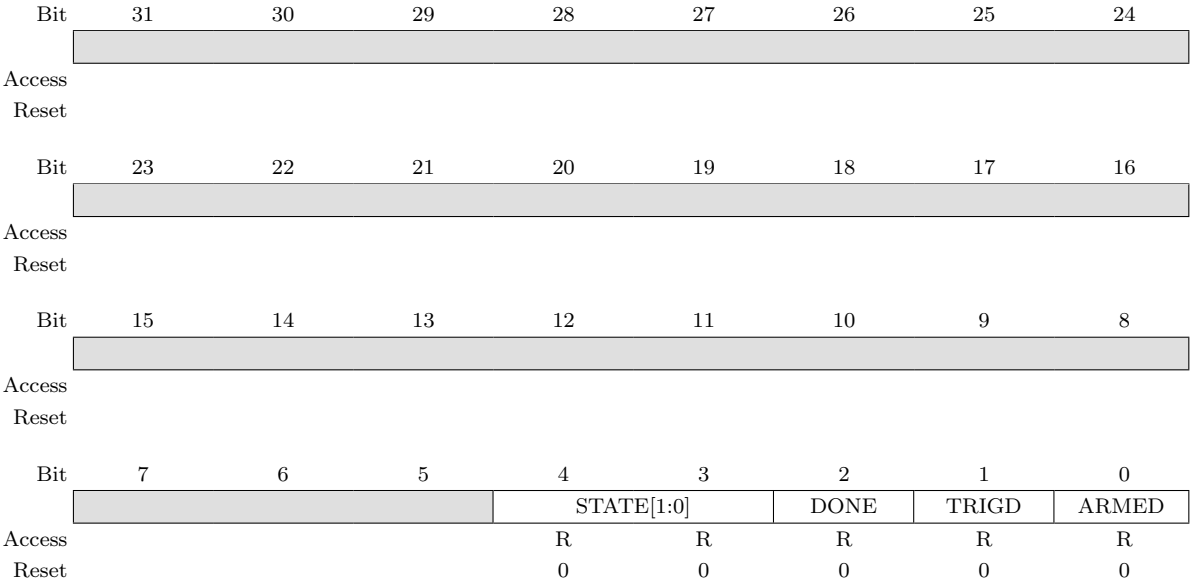
The map therefore holds $8 + 4 + 2a$ control registers followed by $a \times$G_SAMP_BUFF_DEPTH buffer registers. For the stock build that is 20 control registers and 8192 buffer registers, ending at byte offset 0x8050.

The AXI4Lite decoder *truncates* an address it cannot decode rather than rejecting it, so a misaligned or out of range access silently aliases onto a real register. Both supplied drivers check alignment on the host side and the UART wrapper checks it in hardware, because two registers of the input block act on the write rather than on the data: a stray write to ARM_FT would arm the core, and one to DISARM would throw away a capture it was part way through.

7.1 STATUS

Name: STATUS
Register Set: Output block
Offset: `0x00`
Reset: `0x00000000`
Property: Read-only

Status register. It reports the current state of the ILA core. The ARMED, TRIGD and DONE bits are synchronised into the AXI4Lite domain with two flops and so lag the capture domain slightly. STATE is the raw internal state, is not synchronised, and is provided for debugging.



Bits 4:3 – STATE[1:0] Internal state of the capture state machine, not synchronised to the AXI4Lite domain.

Value	Description
2b00	IDLE, waiting to be armed
2b01	ARMED, capturing, trigger not yet seen
2b10	TRIGD, trigger seen, capturing post-trigger samples
2b11	DONE, capture complete, buffer readable

Bit 2 – DONE Set when the capture has completed and the sample buffer holds a full window.
Bit 1 – TRIGD Set when the trigger has fired and the core is capturing its post-trigger samples.
Bit 0 – ARMED Set while the core is armed and capturing.

7.2 MGCKEY

Name: MGCKEY
Register Set: Output block
Offset: `0x04`
Reset: `0xB01DFACE`
Property: Read-only

Magic key register. It reads back a fixed constant, and is the cheapest way for a host to confirm that it is talking to a LibreILA core at the base address it was given, before it trusts anything else in the map.

The constant is the same in every build, deliberately: it identifies the core *type*, never the individual core. Telling one instance from another is the job of UID at `0x14`, and the two are kept apart so that a host can still recognise a core whose UID it has never been told.

Bit	31	30	29	28	27	26	25	24
	MGCKEY[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	1	0	1	1	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	MGCKEY[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	1	1	1	0	1

Bit	15	14	13	12	11	10	9	8
	MGCKEY[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	1	1	1	1	1	0	1	0

Bit	7	6	5	4	3	2	1	0
	MGCKEY[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	1	1	0	0	1	1	1	0

Bits 31:0 – MGCKEY[31:0] Constant `0xB01DFACE`.

7.3 SAMP_CLK_FREQ

Name: SAMP_CLK_FREQ
Register Set: Output block
Offset: `0x08`
Reset: `G_SAMP_CLK_FREQ`
Property: Read-only

Sampling clock frequency in Hz, as given by `G_SAMP_CLK_FREQ` at synthesis. Nothing inside the core uses it; it is here so that a host can turn a sample index into a time and label the waveform axis without being told the clock rate separately.

Bit	31	30	29	28	27	26	25	24
	SAMP_CLK_FREQ[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	SAMP_CLK_FREQ[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	SAMP_CLK_FREQ[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	SAMP_CLK_FREQ[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 – SAMP_CLK_FREQ[31:0] Frequency of `samp_aclk` in Hz. Here and below, a reset of `x` means the bit is fixed at synthesis by the corresponding generic.

7.4 WIDTH

Name: WIDTH
Register Set: Output block
Offset: `0x0C`
Reset: `G_PROBE_WIDTH`
Property: Read-only

Total number of probed bits. Together with DEPTH this is everything a host needs in order to derive the rest of the register map, since the lane count and the stride a both follow from it.

Bit	31	30	29	28	27	26	25	24
	WIDTH[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	WIDTH[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	WIDTH[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	WIDTH[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 – WIDTH[31:0] `G_PROBE_WIDTH`, the width of the probe word in bits.

7.5 DEPTH

Name: DEPTH
Register Set: Output block
Offset: `0x10`
Reset: `G_SAMP_BUFF_DEPTH`
Property: Read-only

Depth of the sample buffer in samples. Always a power of two, so a host can take its logarithm to get the

width of a sample index.

Bit	31	30	29	28	27	26	25	24
	DEPTH[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	DEPTH[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	DEPTH[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	DEPTH[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 – DEPTH[31:0] `G_SAMP_BUFF_DEPTH`, the number of samples the buffer holds.

7.6 UID

Name: UID
Register Set: Output block
Offset: `0x14`
Reset: `G_UID`
Property: Read-only

Identity of this particular core, fixed at synthesis by `G_UID`. Nothing inside the core reads it and no part of the register map moves with it; it exists so that a system carrying several ILAs can be told which one it has reached. Where MGCKEY answers *is this a LibreILA*, UID answers *which LibreILA*.

Splitting the two is what makes discovery work. A host walking a list of base addresses, or of serial ports, checks MGCKEY to know a core is there at all and only then reads UID to find out which one it is. Were the identity folded into MGCKEY instead, a core whose `G_UID` the host had not been told in advance would be indistinguishable from a peripheral that is not an ILA, or from a bus returning zeros.

For that reason every value is legal and none is validated, by the hardware or by either driver. Zero is what a core built without a `G_UID` reports and means no more than *unset*.

Bit	31	30	29	28	27	26	25	24
	UID[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	UID[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	UID[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	UID[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 – UID[31:0] `G_UID`, the identity this instance was given at synthesis. Bit 31 always reads zero: the generic is a `natural`, whose upper bound is `0x7FFFFFFF`.

7.7 SAMP_BUFF_TRIG_IDX

Name: SAMP_BUFF_TRIG_IDX
Register Set: Output block
Offset: `0x18`
Reset: `0x00000000`
Property: Read-only

Index, within the circular sample buffer, of the sample on which the trigger fired. Read it once DONE is set. Note that this is where the trigger *landed* in the buffer, which is not the same thing as TRIG_POS: TRIG_POS is the number of pre-trigger samples the host asked for, this is the raw buffer index needed to go and find them.

The field is `C_ADDR_WIDTH` bits wide, and the diagram below shows the stock 2048-deep build where that is 11 bits. Bits above `C_ADDR_WIDTH` always read zero.

Bit	31	30	29	28	27	26	25	24
Access								
Reset								

Bit	23	22	21	20	19	18	17	16
Access								
Reset								

Bit	15	14	13	12	11	10	9	8
						TRIG_IDX[10:8]		
Access						R	R	R
Reset						0	0	0

Bit	7	6	5	4	3	2	1	0
	TRIG_IDX[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0

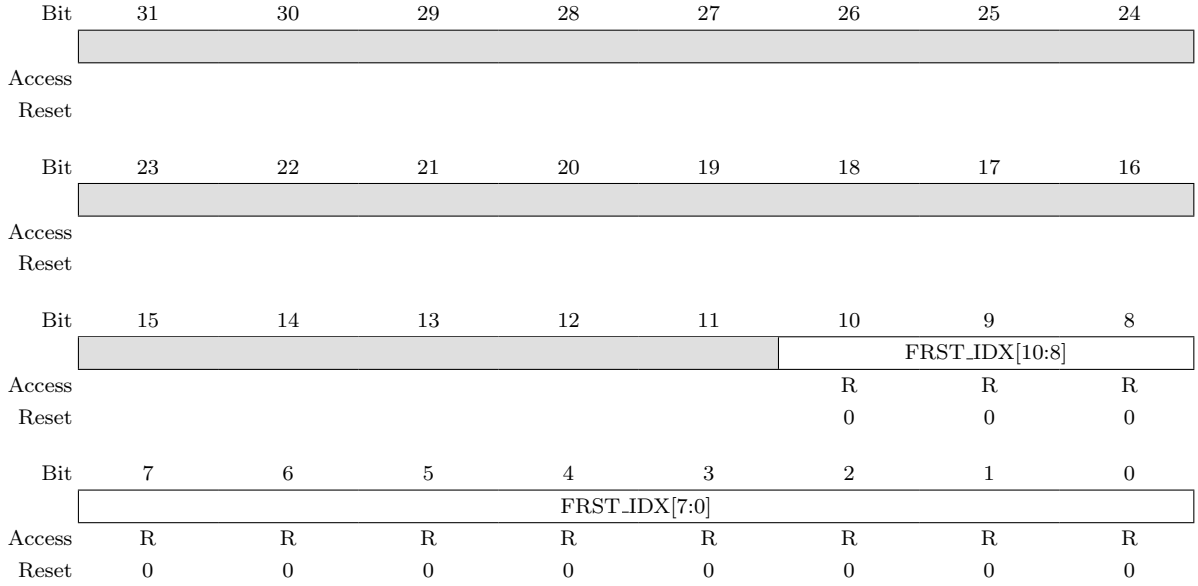
Bits 10:0 – TRIG_IDX Buffer index of the trigger sample, valid once DONE is set.

7.8 SAMP_BUFF_FRST_IDX

Name: SAMP_BUFF_FRST_IDX
Register Set: Output block
Offset: `0x1C`
Reset: `0x00000000`
Property: Read-only

Index, within the circular sample buffer, of the oldest sample of the captured window. Because the buffer wraps, this is where a host has to start reading in order to get the capture in time order: sample i of the

unrolled window lives at buffer index (FRST_IDX + i) mod DEPTH.



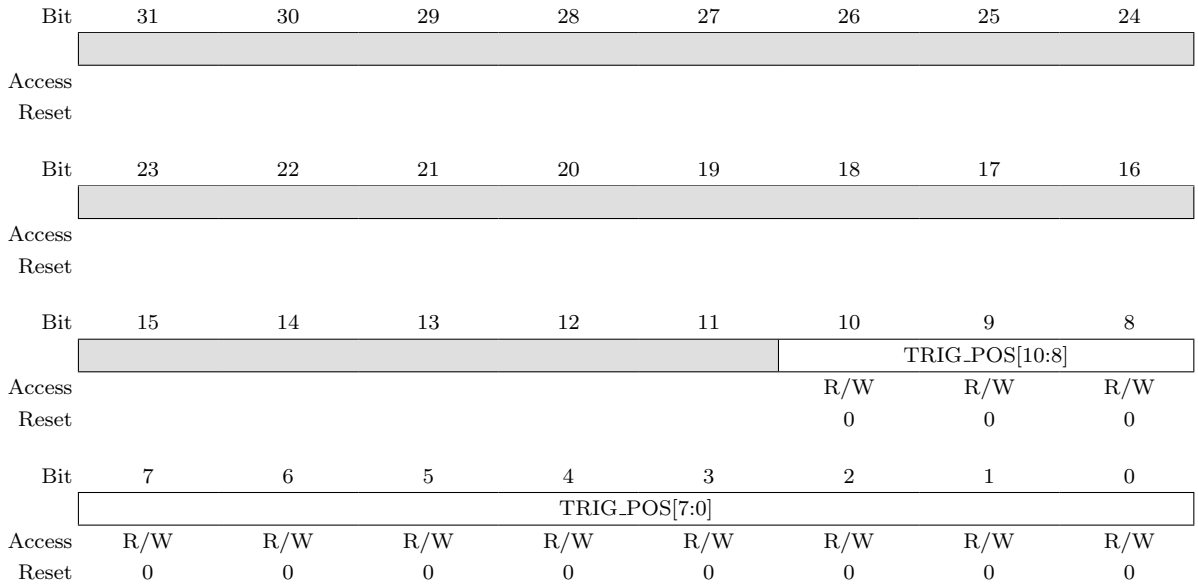
Bits 10:0 – FRST_IDX Buffer index of the oldest sample, valid once DONE is set.

7.9 TRIG_POS

Name: TRIG_POS
Register Set: Input block
Offset: `0x20`
Reset: `0x00000000`
Property: Read/Write

Trigger position, in samples. It is the number of samples of history kept ahead of the trigger, so the core captures for `DEPTH - TRIG_POS - 1` samples after the trigger fires before going to DONE. Zero puts the trigger at the very start of the window, `DEPTH`−1 puts it at the end.

Write this before arming. The field is `C_ADDR_WIDTH` bits wide; the diagram shows the stock 2048-deep build.



Bits 10:0 – TRIG_POS Number of pre-trigger samples.

7.10 ARM_FT

Name: ARM_FT
Register Set: Input block
Offset: 0x24
Reset: 0x00000000
Property: Write-only

Arm and force-trigger register. **Any** write to this register acts; the value written is ignored entirely. A write while the core is IDLE or DONE arms it and starts the capture, and a write while it is already armed forces the trigger immediately, which is how a capture is recovered when the configured condition never occurs.

Configure TRIG_POS, TRIG_CFG, TRIG_COND and TRIG_MASK before writing here.

Bit	31	30	29	28	27	26	25	24
	ARM_FT[31:24]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	ARM_FT[23:16]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
	ARM_FT[15:8]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	ARM_FT[7:0]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 – ARM_FT[31:0] The write itself is the command, the data is a don't care.

7.11 TRIG_CFG

Name: TRIG_CFG
Register Set: Input block
Offset: 0x28
Reset: 0x00000000
Property: Read/Write

Trigger configuration. ANDOR chooses how the mask-enabled bits are reduced to a single condition, and EDGE and FALLING then decide what counts as a trigger on that condition. The reset value of zero gives an AND reduction in level mode.

These bits are ignored when the core is built with `G_EXTERNAL_TRIG` set to 1, where the trigger is always the rising edge of `i_ext_trig`.

Bit	31	30	29	28	27	26	25	24
Access								
Reset								

Bit	23	22	21	20	19	18	17	16
Access								
Reset								

Bit	15	14	13	12	11	10	9	8
Access								
Reset								

Bit	7	6	5	4	3	2	1	0
						FALLING	EDGE	ANDOR
Access						R/W	R/W	R/W
Reset						0	0	0

Bit 2 – FALLING Edge polarity. Only read when EDGE is 1.

Value	Description
0b0	Trigger on the rising edge of the condition
0b1	Trigger on the falling edge of the condition

Bit 1 – EDGE Level or edge sensitivity.

Value	Description
0b0	Trigger while the condition is true
0b1	Trigger on a transition of the condition

Bit 0 – ANDOR Reduction of the mask-enabled bits.

Value	Description
0b0	AND, the condition holds once every enabled bit matches
0b1	OR, the condition holds as soon as any enabled bit matches

7.12 DISARM

Name: DISARM
Register Set: Input block
Offset: `0x2C`
Reset: `0x00000000`
Property: Write-only

Disarm register. **Any** write to this register acts; the value written is ignored entirely, the same way ARM_FT works. A write while the core is ARMED or TRIGD cancels the capture and returns it to IDLE, so a capture waiting on a condition that never comes, or one started by the wrong condition, can be taken back without resetting the core.

Cancelling from TRIGD abandons the post-trigger samples still owed. The samples already taken stay in the buffer, but SAMP_BUFF_TRIG_IDX goes back to zero with the state, so nothing that is read out afterwards claims to be a capture.

A write while the core is IDLE does nothing, and a write while it is DONE is *ignored*: DONE means a finished capture is sitting in the buffer waiting to be read, and returning to IDLE would clear the trigger index that is needed to unwrap it. Disarm cancels a capture, it does not discard a finished one — arm again to start the next.

A trigger arriving in the same cycle as a disarm wins, so a race between the two leaves the core in TRIGD rather than IDLE. A second disarm then cancels it.

Bit	31	30	29	28	27	26	25	24
	DISARM[31:24]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	DISARM[23:16]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
	DISARM[15:8]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	DISARM[7:0]							
Access	W	W	W	W	W	W	W	W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 – DISARM[31:0] The write itself is the command, the data is a don't care.

7.13 TRIG_COND(n) and TRIG_MASK(n)

Name: TRIG_COND(n) / TRIG_MASK(n)
Register Set: Input block
Offset: `0x30` + $4n$ / `0x30` + $4a + 4n$
Reset: `0x00000000`
Property: Read/Write

The trigger vector, one bit per probed bit, spread over a registers each. Word n carries probe bits $32n+31$ downto $32n$, which is exactly the layout one sample of the buffer uses, so a captured sample can be written straight back as a condition.

For probe bit i, the TRIG_COND bit is the value that bit has to take in order to match, and the TRIG_MASK bit decides whether it takes part at all, 1 enabling it and 0 making it a don't care. Bits above `G_PROBE_WIDTH` up to the stride boundary are padding; leave their mask bits at 0.

Bit	31	30	29	28	27	26	25	24
	TRIG_COND(n)[31:24] / TRIG_MASK(n)[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	23	22	21	20	19	18	17	16
	TRIG_COND(n)[23:16] / TRIG_MASK(n)[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
	TRIG_COND(n)[15:8] / TRIG_MASK(n)[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	TRIG_COND(n)[7:0] / TRIG_MASK(n)[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 Trigger condition and mask for probe bits $32n + 31$ downto $32n$.

7.14 SAMP_BUFF(k,l)

Name: SAMP_BUFF(k,l)
Register Set: Sample buffer
Offset: `0x30` + $8a + 4(ak + l)$
Reset: —
Property: Read-only

The capture itself. Sample k occupies a consecutive registers, of which the first `C_N_LANES` carry the probe word, lane l holding probe bits $32l + 31$ downto $32l$. The last lane is zero padded in its upper bits, and the registers between `C_N_LANES` and the stride boundary are padding that reads back as zero.

That padding is what keeps addressing cheap: with a a power of two, a sample index becomes an address with a shift instead of a multiply. It costs bus address space and not RAM, since the buffer itself is only `C_N_LANES` lanes wide.

Read the buffer only once DONE is set, and start from SAMP_BUFF_FRST_IDX to get the window in time order.

Bit	31	30	29	28	27	26	25	24
	SAMP_BUFF(k,l)[31:24]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	23	22	21	20	19	18	17	16
	SAMP_BUFF(k,l)[23:16]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	15	14	13	12	11	10	9	8
	SAMP_BUFF(k,l)[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bit	7	6	5	4	3	2	1	0
	SAMP_BUFF(k,l)[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	x	x	x	x	x	x	x	x

Bits 31:0 Probe bits $32l + 31$ downto $32l$ of sample k. The contents are undefined until a capture has completed.

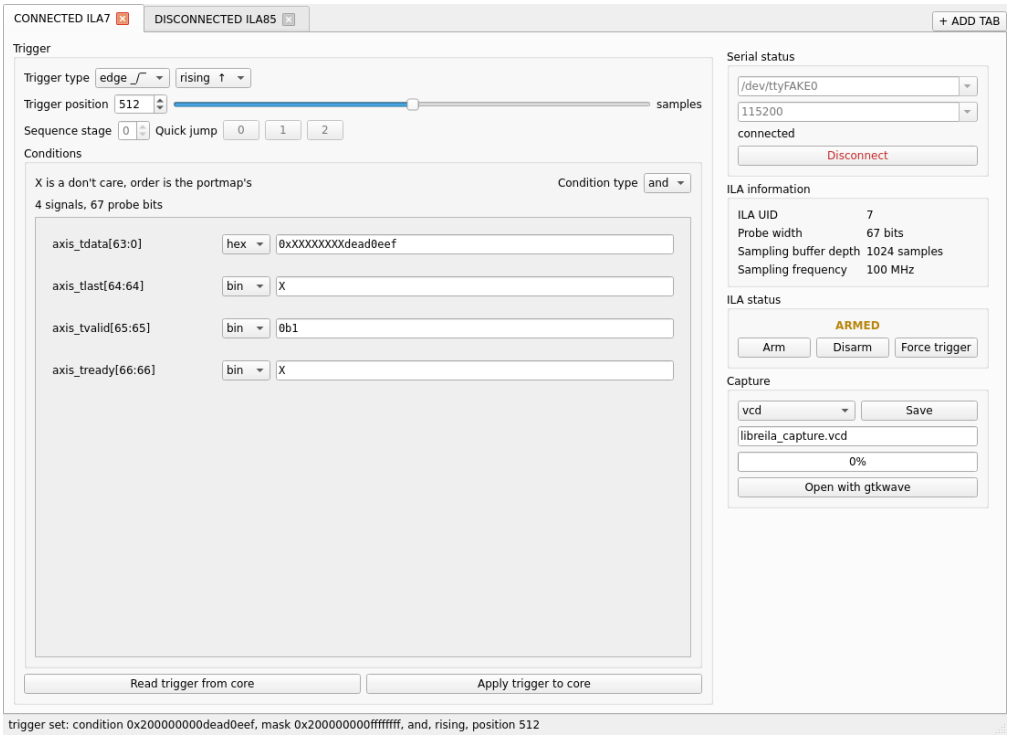


Figure 3: Image of the GUI showing ILA in armed state.

8 AXI4Lite UART interface

`libre_ila_uart` wraps the core with a UART and a small packet parser, so that a design with no processor, or with no spare AXI master, can still be debugged from a PC over a two-wire serial link. The wrapper instantiates the core and carries all of its generics, adding the FIFO depths and the baud rate. The register map seen from the PC is exactly the map of Table 6, reached by address rather than through any command protocol.

The supplied python driver speaks the packet format, splits long transfers, unrolls the circular buffer and writes a VCD with the signals named from `portmap.csv`, which any waveform viewer will open.

TODO: Add the GUI subsection here once the GUI is released, with screenshots of a capture being configured and read back.

8.1 Watchdog and error handling

Two safety nets keep a bad link from wedging the wrapper.

A **watchdog** counts while the wrapper is in any non-idle state and is cleared whenever a byte makes progress. After `G_AXIL_CLK_FREQ` counts, that is one second without progress, the wrapper resets to idle and abandons whatever it had parsed. A timeout therefore loses the whole transaction rather than half applying it, and the host recovers by restarting from its sync byte; since the idle state discards every byte that is not a sync byte, the link resynchronises on its own.

Each request is also **sanity checked** once its base address is in. The address must be four byte aligned, because the core's decoder truncates rather than rejects and a misaligned write could land on ARM_FT. The receive FIFO is checked for overflow, since the UART writes into it unconditionally. A request that fails either check is answered but never put on the AXI bus: a read returns the promised number of zero words, and a write still has its payload drained, so the link stays in step.

8.2 UART Packet Format

The PC is the master of the link. Every transaction is one request packet from the PC and one response packet from the wrapper, both beginning with a sync byte and a six byte header.

Field	Size	PC to wrapper
SYNC	1 byte	`0x55`
R/W	1 bit	1 write, 0 read
#words	7 bits	word count, 1 to 127
Base address	4 bytes	byte address, 4 byte aligned
Write data	4 bytes each	#words words, write only

Table 7: Request packet

Field	Size	Wrapper to PC
SYNC	1 byte	`0xAA`
valid	1 bit	1 accepted, 0 refused
#words	7 bits	echoed word count
Base address	4 bytes	echoed base address
Read data	4 bytes each	#words words, read only

Table 8: Response packet

Points worth knowing about the framing:

- Every field wider than a byte goes **MSB first**, both the base address and the data words. The header is six bytes, the payload 4× #words bytes.
- The R/W bit and the word count **share one byte**: bit 7 is R/W, bits 6 downto 0 are the count. The response byte is the same shape, with *valid* in bit 7.
- **One packet moves at most 127 words.** Longer transfers are split across packets with the base address advancing by four per word. This is routine and not an edge case: reading the stock 2048-sample buffer at stride 4 is 8192 words, that is 65 packets.
- **A write is acknowledged too.** The response header goes out as soon as the address has been parsed, before the write data has even arrived, so a host that does not drain the six byte acknowledgement will desynchronise the next transaction. Only a read is followed by data words.
- Because the write header precedes the write payload, an overflow during that payload is reported on the *next* packet's header. Overflow detection is best effort: valid = 0 means bytes were dropped, valid = 1 does not promise that none were.

9 Additional references

- **Repository and full documentation** — `github.com/yashparitkar/libreila`. The top level README is the authoritative description of the register map, the trigger vector and the packet format.
- **LibreILA User Manual** — the full instantiation, generation and debug procedure, of which this datasheet is a summary.
- **License** — CERN Open Hardware Licence (OHL) Version 2, Permissive. See the `LICENSE` file in the repository.
- **Microchip G238606** — PolarFire family fabric user guide, for the LSRAM block and its independent read clock, which the readout path relies on.
- **ARM IHI 0022** — AMBA AXI and ACE protocol specification, for the AXI4Lite slave port.
- **VHDL Style Guide (VSG)** — recommended for reformatting the generated sources before they are committed to a project.

10 Device utilization and performance

Resource usage scales with the two generics that describe the capture, and with nothing else. The storage requirement is exactly

$$\text{bits} = \texttt{G\_SAMP\_BUFF\_DEPTH} \times \texttt{C\_N\_LANES} \times 32$$

that is, the probe word rounded up to a whole number of 32-bit lanes. Note that the stride padding used on the bus costs address space only and is not stored, so a 67-bit probe occupies three lanes in RAM and not four. For the stock build that is $2048 \times 3 \times 32 = 196{,}608$ bits, which maps onto PolarFire LSRAM blocks.

Logic usage is dominated by the trigger comparator, which is combinational and carries two bits of configuration per probed bit, plus the sample counter and the small AXI4Lite slave state machine. The UART wrapper adds its two FIFOs, sized by `G_UART_RX_FIFO_DEPTH` and `G_UART_TX_FIFO_DEPTH`, the UART itself and the packet parser.

TODO: Instantiate the core on the PolarFire SoC at several probe widths and buffer depths, and fill in the measured LUT, DFF and LSRAM counts along with the achieved fmax for both clock domains.

Probe width	Depth	LUTs	DFFs	LSRAM	fmax
67	2048	—	—	—	—
32	1024	—	—	—	—
128	4096	—	—	—	—

Table 9: Device utilization, PolarFire SoC. To be measured.